

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



SOFTWARE ARCHITEKTUR

Themen in dieser Vorlesung



Lernziele

- Sinn und Aufgabe von Softwarearchitektur verstehen
- Architekturdefinitionen kennen und verstehen
- Darstellungen von Architektur kennen
- Aufgabenfelder eines Software Architekten kennen



Architektur: Grundbegriffe

Komponente

*component – One of the parts that make up a system.
A component may be hardware or software and may be subdivided
into other components. IEEE Std 610.12 (1990)*

- Eine Komponente ist somit ein **Bestandteil** eines Systems
- Eine Komponente bietet ihrer Umgebung eine **Menge von Diensten** an, die über eine wohldefinierte **Schnittstelle** genutzt werden können
- Zu einer Komponentenbeschreibung können auch die benötigten **Dienste anderer Komponenten** gehören

Architektur: Grundbegriffe

Schnittstelle

- Metapher „Schnittstelle“: Schneidet man einen Gegenstand in zwei Teile, so entstehen zwei Oberflächen, die **spiegelsymmetrisch** sind.
- Was durch einen Schnitt von selbst entsteht, die beiden exakt zueinander passenden Oberflächen, muss in der Technik **hergestellt** werden.

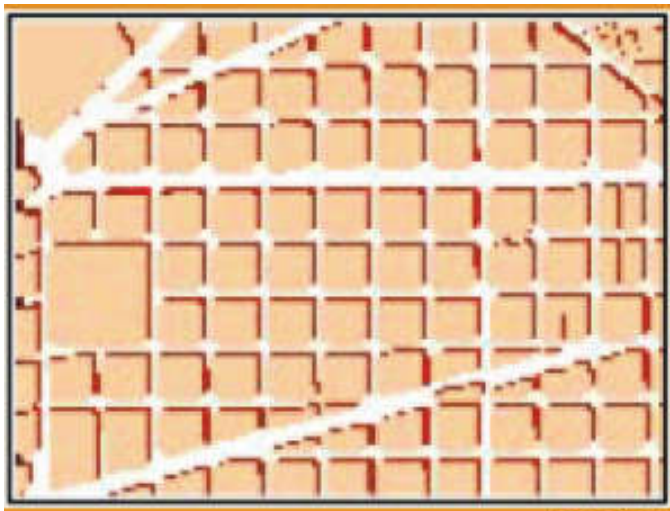
Das englische Wort „**interface**“ ist darum der Realität näher.

Schnittstellen = Festlegungen, die die Kombinierbarkeit sicherstellen sollen.

- Achtung: **Adapter** (syn.: **Konnektoren**) sind **keine** Schnittstellen.
- Die Programmiersprache **Java** erlaubt es, Schnittstellen explizit (als **Interfaces**) zu beschreiben.

Architektur

Architektur



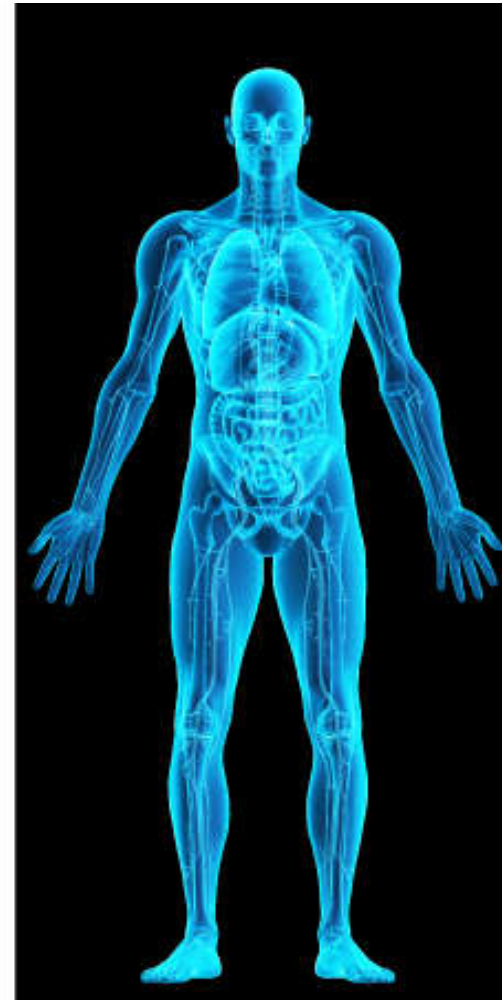
Architektur



Architektur



Funktionsweise verstehen durch Konzentration auf Struktur



Aus: Len Bass, Paul Clements, Rick Kazman: Software Architecture in Practice, 3rd Edition, Addison-Wesley Professional, 2012.

Struktur

Bau, Aufbau, Gefüge, Gliederung

von lat. **structura** „*ordentliche*

Zusammenfügung, Ordnung, Aufbau, Bauart“

zu **struere** „*übereinander schichten, aneinander
fügen, aufbauen, errichten*“

Hierarchische Gliederung

- *Miller (1956)*: Ein Mensch kann nur Systeme überblicken, die aus wenigen, **typisch etwa sieben Teilen** bestehen
- Mehr als sieben: Gegenstände nacheinander betrachten und ihren Zusammenhang systematisch entwickeln
- Programme (Systeme) bestehen typisch aus **einigen bis vielen tausend** Deklarationen und Anweisungen. Sie müssen hierarchisch organisiert werden, um für uns überschaubar zu sein.

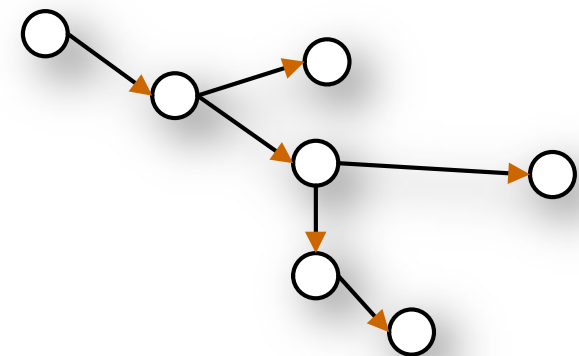
Hierarchische Gliederung

- Uhrmacher-Parabel: Nur dank (und mit) Abstraktion können wir sehr komplexe Objekte herstellen.
- Nur kleine Programme kann man direkt nach den Anforderungen codieren; alle anderen müssen **schrittweise entwickelt** werden.
- Durch die sinnvolle Gliederung (und nur durch sie) entsteht die Möglichkeit zur **Abstraktion**.

Software-Struktur

Softwarestruktur: Alle vom Standpunkt des Softwareingenieurs statisch beobachtbaren Einheiten und Beziehungen aus dem Entwurf und der Implementierung eines Softwaresystems.

- Sicht auf den statischen Aufbau eines Softwaresystems
 - seine Bausteine und deren statischen, inneren Aufbau.
- Wird festgelegt auf verschiedenen Ebenen. Beispiele:
 - Einzelnen Anweisungen im Quelltext
 - Methoden
 - Klassen
 - Komponenten
 - Subsysteme



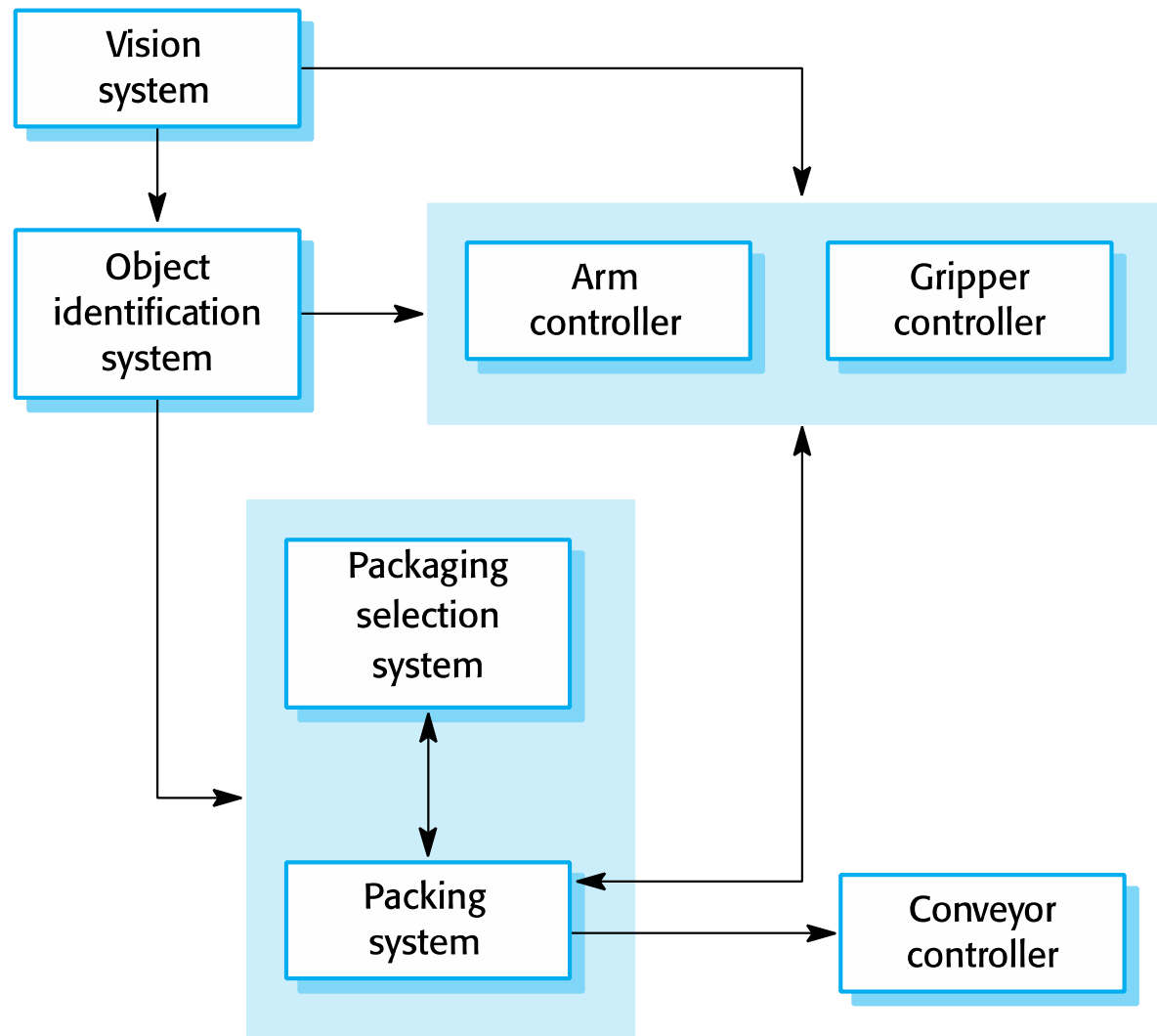
Software-Architektur

The software architecture of a program or computing system is the structure of structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them. (Bass / Clements / Kazman (1998))

- Software-Architektur ist also eine Teilmenge der Software-Struktur
 - In der Regel sind es die höchsten Abstraktionsebenen der Software-Struktur
- Wort-Herkunft: *architectura* (lateinisch) „*Die Kunst und die Wissenschaft, Bauten und Strukturen zu entwerfen*“

Beispiel:

Architektur der Steuerung eines Packroboters



Womit befaßt sich Software-Architektur?

- Architektur will verstehen, wie der **Aufbau** eines Software-Systems sein sollte und entwirft diese **Gesamtstruktur**
- Architektur ist kritische Verbindung zwischen Entwurf/Implementierung und Anforderungen, da sie die **Hauptkomponenten des Systems** und die **Beziehungen** zwischen diesen ermittelt
- Das Ergebnis ist ein Architektur-Modell (kurz: "**System-Architektur**"), das beschreibt wie das System als **Menge miteinander kommunizierender Komponenten** aufgebaut ist

Womit befaßt sich Software-Architektur?

- Architektur ist

Architektur legt also fest:

- Aus welchen Komponenten besteht das System?
- Haben Sie eine Beziehung? Wie kommunizieren sie? Untereinander und mit der Außenwelt?
- Welche Schnittstellen?
 - Es geht nicht um die Details der *inneren* Funktionsweise der Elemente in der Struktur, sondern den Blick *von außen* auf diese Elemente
- Es geht auch darum, nach welchen Prinzipien das System und seine Komponenten weiterentwickelt werden sollen

Ziel von (Software-)Architektur

A good design is one that balances trade-offs to minimize the total cost of the system over its entire lifetime. (Coad / Yourdon, 1991)

Ziel von Architektur ist **richtige Systemaufteilung** –
über die gesamte Lebenszeit hinweg!

Gibt es immer eine Software-Architektur?

Schauen wir auf die Definition!

Kann etwas keine Struktur haben? Oder ist nicht vielmehr eine ungeplante oder zufällige Struktur auch Struktur?

➤ Jedes System hat eine Architektur!

- Egal ob explizit ausgeführt
- Egal ob dokumentiert
- Egal ob gut oder schlecht

Bedeutung der Software-Architektur für die Software-Entwicklung

- Fördert **Verständnis und Klarheit** über Gesamtsystem und dessen Komplexität durch geeignete Abstraktionen
- Unterstützt **Wiederverwendung** auf unterschiedlichen Ebenen des Systems
- Liefert einen **Bauplan** für die konkrete Entwicklung und Realisierung des Systems
- Stellt den **Rahmen für Erweiterungen** des Systems zur Verfügung
- Fördert die **Kommunikation mit Stakeholdern**
- Unterstützt **Systemanalyse** und Erkennung **potenzieller Risiken**



Bedeutung der Software-Architektur für das Business (1/2)

The Business Model Canvas

Designed for:

Designed by:

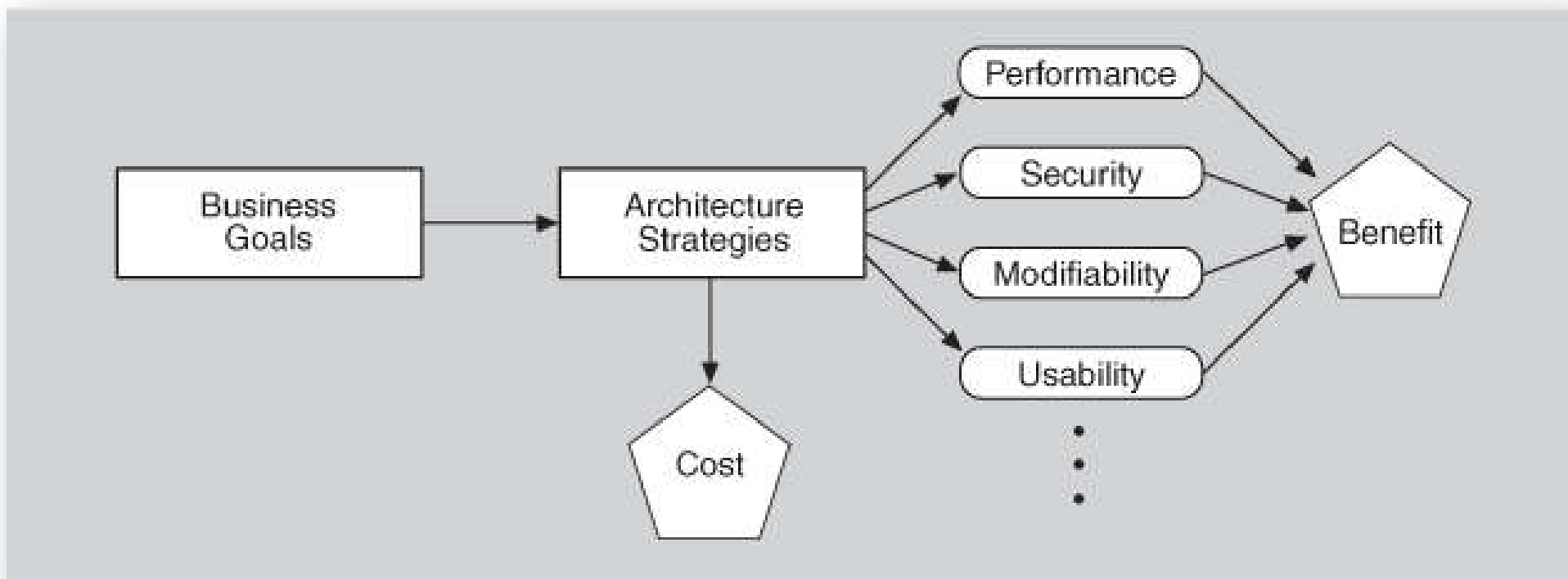
On:

Iteration:

Key Partners  Who are our Key Partners? Who are our key suppliers? Which Key Resources are we acquiring from partners? Which Key Activities do partners perform? acquire resources optimize and economize reduce risk and uncertainty optimize of particular resources and activities	Key Activities  What Key Activities do our Value Propositions require? Our Distribution Channels? Customer Relationships? Revenue streams? Production Distribution Problem Solving Platform/Network	Value Propositions  What value do we deliver to the customer? Which one of our customer's problems are we helping to solve? What bundles of products and services are we offering to each Customer Segment? Which customer needs are we satisfying? Value Propositions Design Production Distribution Customer Support "Selling the Ice Cream" Design Brand/Status Price User Experience Risk Reduction Accessibility Complementary Products	Customer Relationships  What type of relationship does each of our Customer Segments expect us to establish and maintain with them? Which ones have we established? How are they integrated with the rest of our business model? How costly are they? Relationships Personal Assistance Individual Personal Assistant Self Service Automated Services Communities Co-creation	Customer Segments  For whom are we creating value? Who are our most important customers? Mass Market Niche Market Segmented Individual Multi-sided Platform
Key Resources  What Key Resources do our Value Propositions require? Our Distribution Channels? Customer Relationships? Revenue Streams? Human or Material Assets Physical Intellectual (brand names, copyrights, data) Financial		Channels  Through which Channels do our Customer Segments want to be reached? How are we reaching them now? How are our Channels integrated? Which ones work best? Which ones are most cost-efficient? How are we integrating them with customer routines? Channels Strategy 1. Awareness How do we communicate about our integrated product/service model? 2. Evaluation How do we help customers evaluate our integrated Value Proposition? 3. Purchase How do we allow customers to purchase goods and services online? 4. Delivery How do we deliver Value Proposition components? 5. After sales How do we help customers purchase additional support?		
Cost Structure  What are the most important costs inherent in our business model? Which Key Resources are most expensive? Which Key Activities are most expensive? Cost Structure Strategy Cost Structure (based on key resources, key price value propositions, maximum, minimum, maximum, minimum, maximum, minimum) Value Drivers (based on value creation, production value propositions) Fixed Costs (salaries, rent, utilities) Variable Costs Economies of scale Economies of scope		Revenue Streams  For what value are our customers really willing to pay? For what do they currently pay? How are they currently paying? How would they prefer to pay? How much does each Revenue Stream contribute to overall revenues? Revenue Streams Asset Sale Usage Fee Subscription Fee Licensing Advertising Sponsorship SaaS (PaaS) Product/Service/Asset Customer segment dependent Platform/Network Regulatory/Compliance Third-Party Revenue Real-time Market		

Bedeutung der Software-Architektur für das Business (2/2)

Architektur bestimmt maßgeblich, ob und wie Geschäftsziele mit den normalerweise begrenzten Ressourcen erreicht werden können.



Bass, Len; Clements, Paul; Kazman, Rick (2013): Software Architecture in Practice. 3. Aufl. Upper Saddle River, NJ: Addison-Wesley (SEI Series in Software Engineering).

Architektur drückt nicht-funktionale Eigenschaften aus

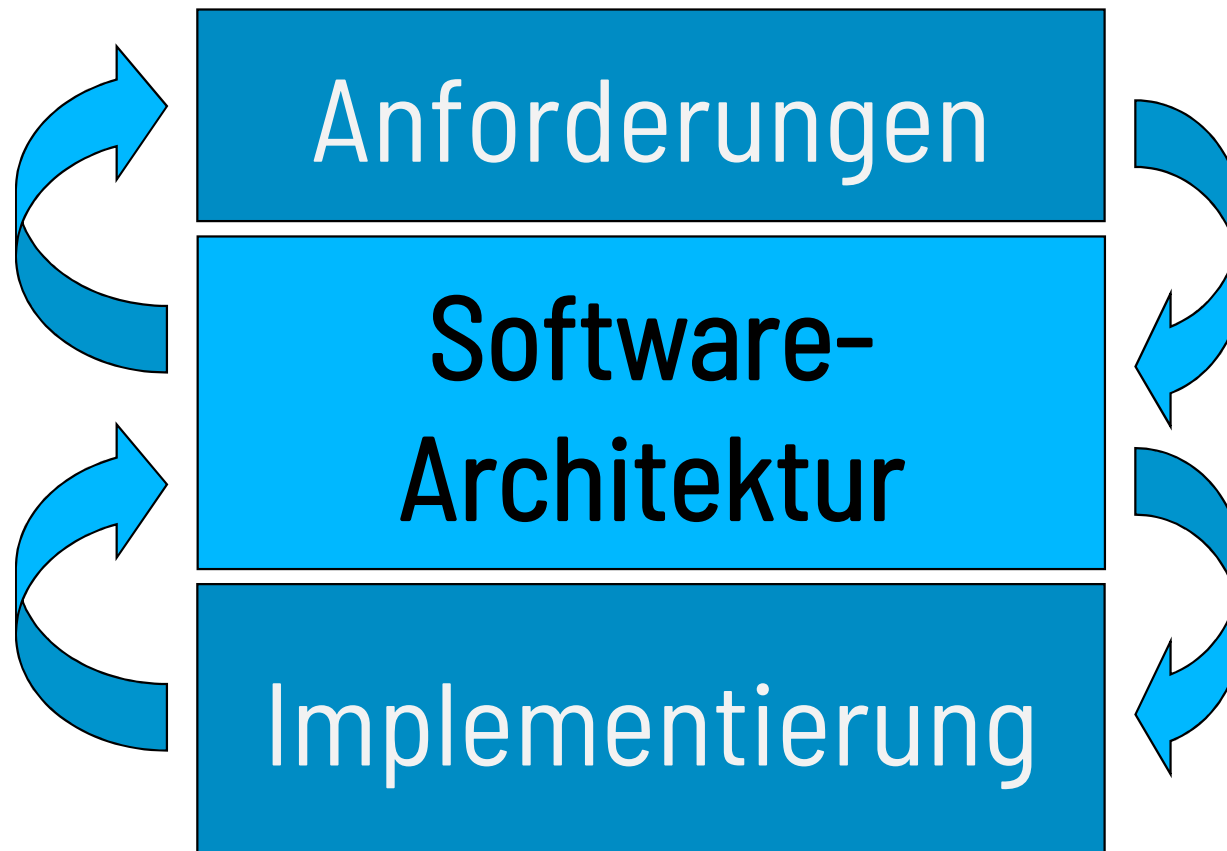
Beispiele für solche nicht-funktionalen Eigenschaften:

- Wartbarkeit (Maintainability)
 - Möglichkeit, ein System weiterentwickeln zu können
- Portierbarkeit (Portability)
 - Anpassbarkeit an neue Hardware und Betriebssysteme
- Performance
 - Ausführungsgeschwindigkeit, Speicherverbrauch
- Sicherheit (Security)
 - Schutz vor Manipulation von Algorithmen und Daten sowie Schutz vor ungewollter Offenlegung von Daten
- Betriebssicherheit (Safety)
 - Verlässlichkeit / Korrektheit der Ergebnisse
- Verfügbarkeit (Availability)
 - Das System bleibt nutzbar

Gestaltung von Architektur im Hinblick auf Systemeigenschaften

- Wartbarkeit (Maintainability)
 - Setze kleine, leicht ersetzbare Komponenten ein.
- Performance
 - Lokalisierere kritische Operationen und minimiere Kommunikation. Benutze eher größere als kleine Komponenten.
- Sicherheit (Security)
 - Verwende eine Architektur mit mehreren Schichten und platziere kritische Güter in den inneren Schichten.
- Betriebssicherheit (Safety)
 - Platziere die ausfall-kritischen Funktionen in einer kleinen Zahl von Subsystemen.
- Verfügbarkeit (Availability)
 - Schließe redundante Komponenten und Mechanismen für Fehlertoleranz ein.

Brückenfunktion der Software-Architektur



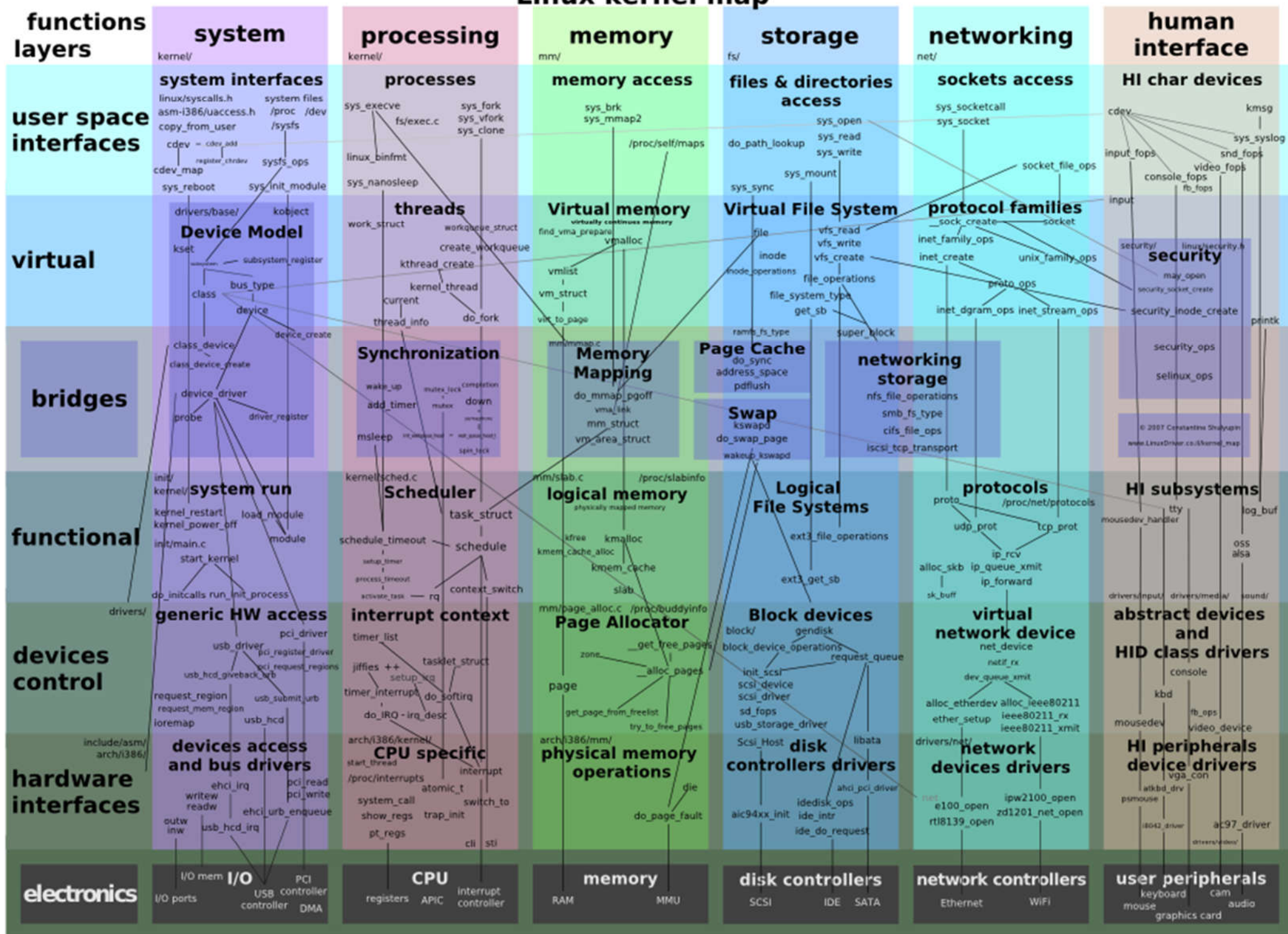
Darstellung von Architektur

- Einfache, informelle Blockdiagramme, die Elemente und ihre Beziehungen zeigen, sind die häufigste Methode zur Darstellung
 - Gleichzeitig fehlt dort schnell Semantik (Klarheit über einheitliche und genaue Bedeutung in der Darstellung) – besonders bei Arten der Beziehung, Eigenschaften der Elemente, ...
 - UML? Viel Semantik, aber eher wenig Möglichkeiten zur Beschreibung größerer Einheiten
 - Es gibt Architektur-Beschreibungssprachen (ADL), diese haben aber keine breite Verwendung
- Letztlich werden Darstellung und Semantik des Modells davon bestimmt, wie das Modell eingesetzt wird.

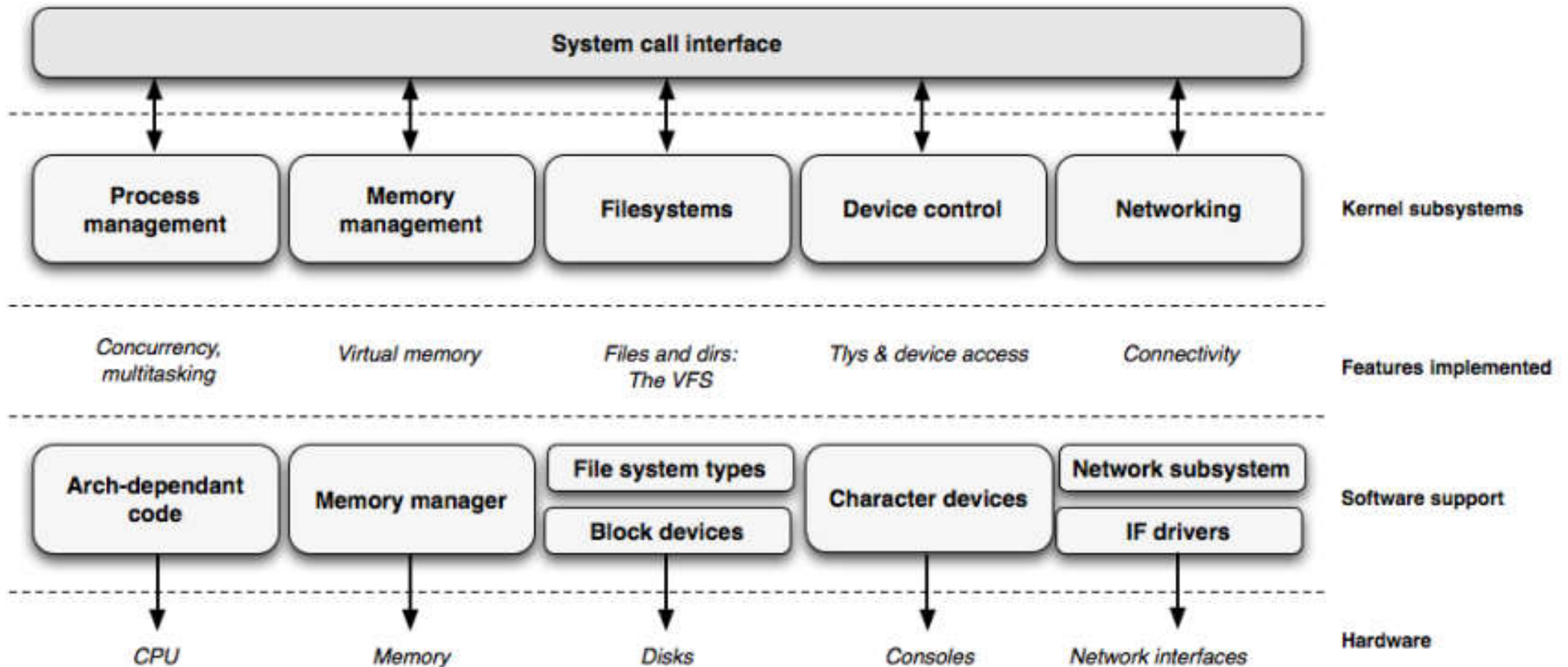
Architekturdarstellung mit „Kästen und Linien“

- **Sehr abstrakt**, da normalerweise Natur von Komponentenbeziehungen oder nach außen sichtbare Eigenschaften von Subsystemen nicht dargestellt werden
- Trotzdem **nützlich** für Kommunikation mit Stakeholdern und für Projektplanung.

Linux kernel map



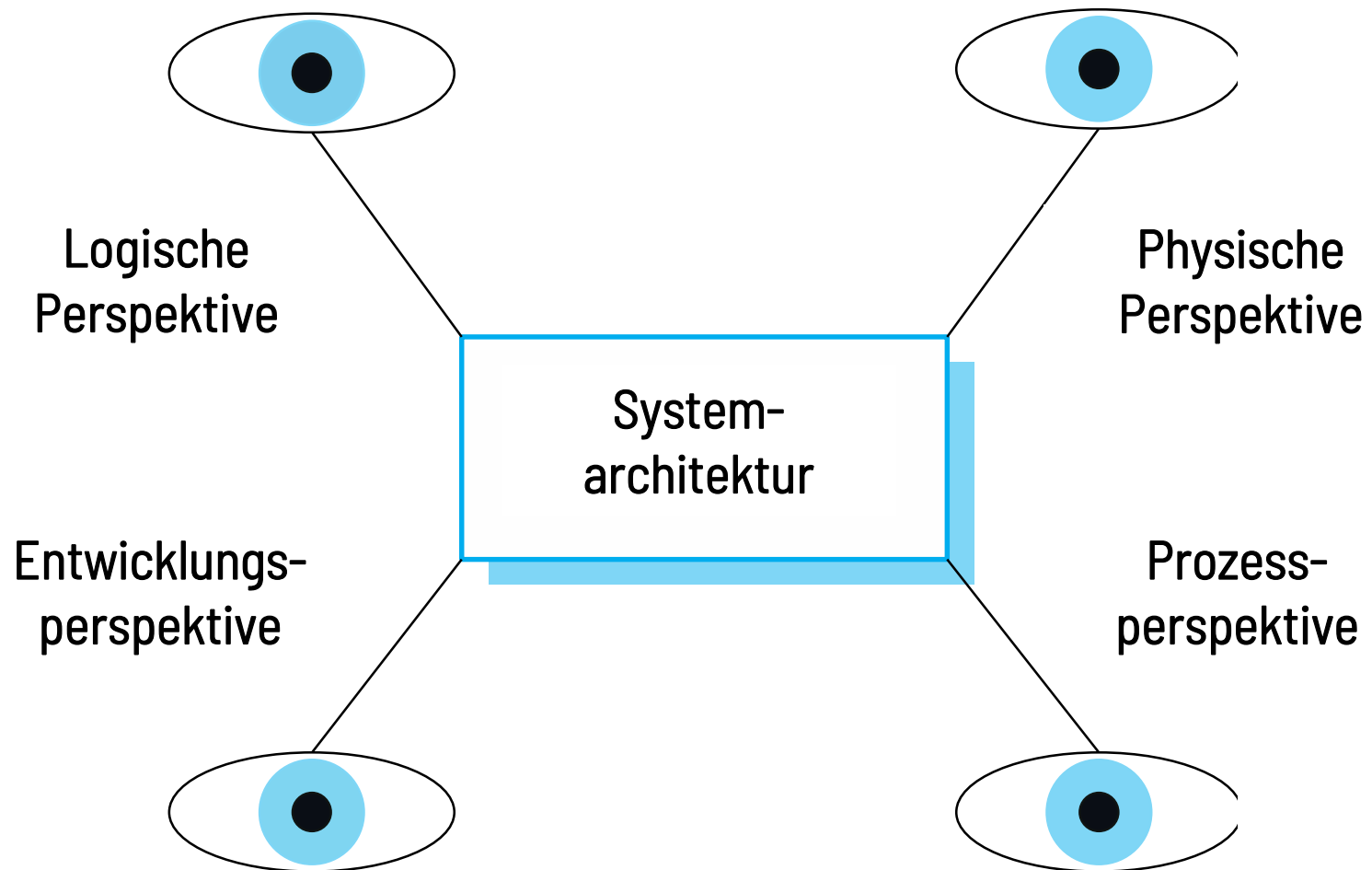
Beispiel: Alternative Darstellung der Linux-Kernel-Architektur



Architekturperspektiven

- Jede Darstellung von Systemarchitektur zeigt nur **eine Perspektive** auf das System:
 - Aufteilung in Module
 - Wie Prozesse zur Laufzeit interagieren
 - Verteilung von Systemkomponenten über ein Netzwerk
- Welche Perspektiven sind nützlich, wenn man eine Systemarchitektur entwirft oder dokumentiert?
- Wie stellt man diese Perspektiven dar?
- Normalerweise benötigt man mehrere Perspektiven für den Entwurf einer Architektur oder deren Dokumentation.

Architekturperspektiven



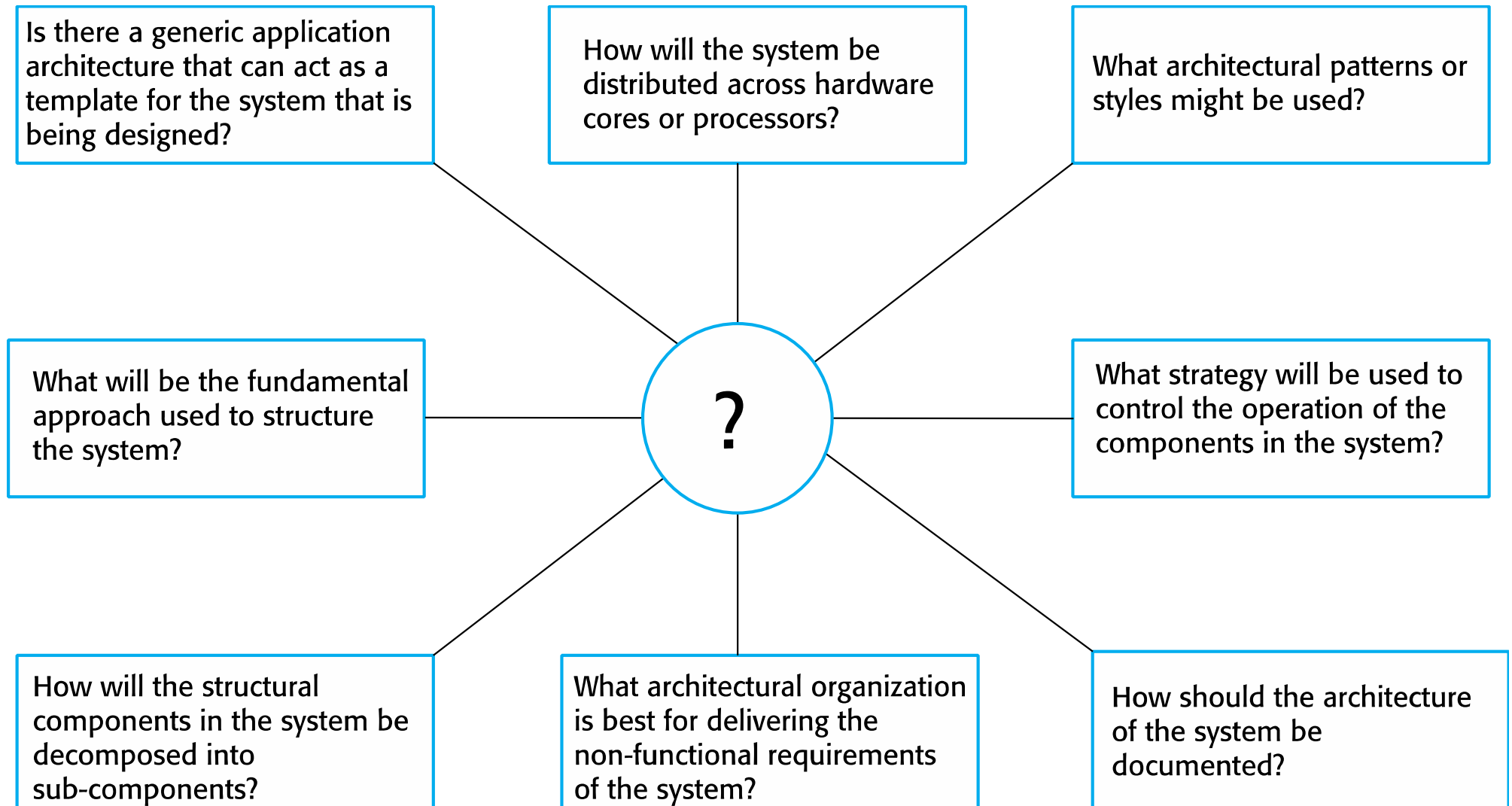
Das 4 + 1-View-Modell für Software-Architektur

- **Logische Perspektive:** Zeigt die Schlüsselabstraktionen des Systems als Objekte oder Klassen.
- **Prozess-Perspektive:** Zeigt, aus welchen interagierenden Prozessen das System zur Laufzeit besteht.
- **Entwicklungs-Perspektive:** Zeigt, wie das System für die Entwicklung aufgeteilt wird.
- **Physische Perspektive:** Zeigt die System-Hardware und wie die Softwarekomponenten über Ausführungseinheiten (Prozessoren) verteilt sind.
- Die Perspektiven werden durch Use Cases oder Szenarien in **Beziehung gesetzt („+1“)**

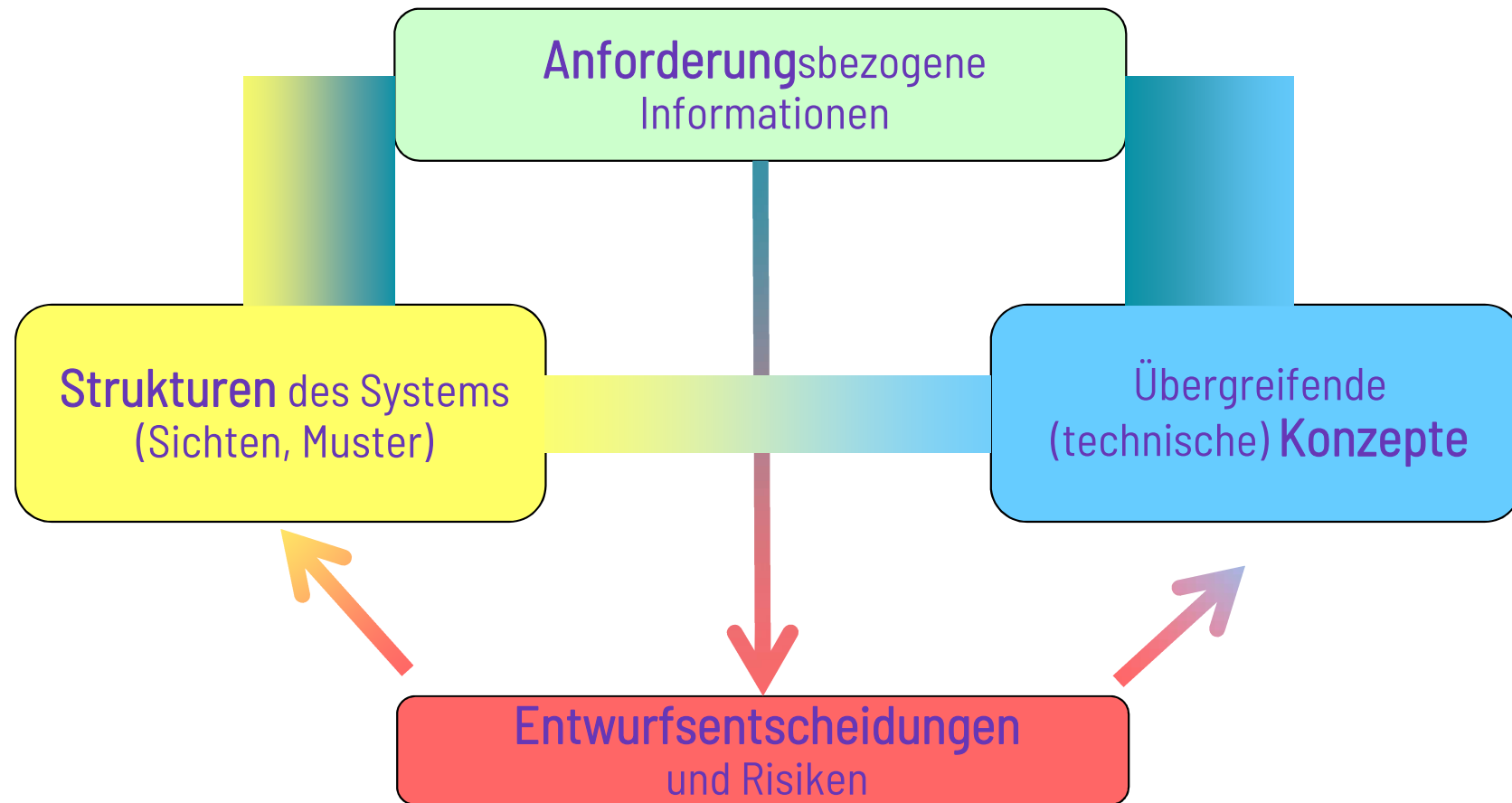
Entwurfsentscheidungen bei Architektur

- Der Entwurf einer Architektur ist ein kreativer Prozess, der vom Typ des Systems abhängt, das entwickelt werden soll.
- Eine Reihe von Entscheidungen in diesem Prozess beeinflussen alles weitere – insbesondere die nicht-funktionalen Eigenschaften des zu entwickelnden Systems.
- Gutes Design ist ein Kompromiss, das bedeutet auch: Man kann nicht alles gleichzeitig haben!

Architektur-Entscheidungen



Architekturdokumentation nach Arc42.de



1. Einführung und Ziele

Aufgabenstellung, Qualitätsziele, eine Kurzfassung der architekturrelevanten Anforderungen (insb. die nichtfunktionalen), Stakeholder.

2. Randbedingungen

Welche Leitplanken schränken die Entwurfsentscheidungen ein?

3. Kontextabgrenzung

In welchem fachlichen und/oder technischen Umfeld arbeitet das System?

4. Lösungsstrategie

Wie funktioniert die Lösung? Was sind die fundamentalen Lösungsansätze?

5. Bausteinsicht

Die statische Struktur des Systems, der Aufbau aus Implementierungsteilen.

6. Laufzeitsicht

Zusammenwirken der Bausteine zur Laufzeit, gezeigt an exemplarischen Abläufen ("Szenarien")

7. Verteilungssicht

Deployment: Auf welcher Hardware werden die Bausteine betrieben?

8. Querschnittliche Konzepte und Muster

Wiederkehrende Muster und Strukturen.

Fachliche Strukturen.

Querschnittliche, übergreifende Konzepte, Nutzungs- oder Einsatzanleitungen für Technologien. Oftmals projekt-/systemübergreifend verwendbar!

9. Entwurfsentscheidungen

Zentrale, prägende und wichtige Entscheidungen.

10. Qualitätsszenarien

Qualitätsbaum sowie dessen Konkretisierung durch Szenarien

11. Risiken

12. Glossar

Wichtige Begriffe.

Struktur der arc42-Schablone

Legende:

anforderungsbezogene
Informationen

Strukturen der Lösung (Sichten)

übergreifende (technische)
Informationen

besonders wichtige Entscheidungen

1. Einführung und Ziele

Aufgabenstellung, Qualitätsziele, eine Kurzfassung der architekturrelevanten Anforderungen (insb. die nichtfunktionalen), Stakeholder.

2. Randbedingungen

Welche Leitplanken schränken die Entwurfsentscheidungen ein?

3. Kontextabgrenzung

In welchem fachlichen und/oder technischen Umfeld arbeitet das System?

4. Lösungsstrategie

Wie funktioniert die Lösung? Was sind die fundamentalen Lösungsansätze?

5. Bausteinsicht

Die statische Struktur des Systems, der Aufbau aus Implementierungsteilen.

6. Laufzeitsicht

Zusammenwirken der Bausteine zur Laufzeit, gezeigt an exemplarischen Abläufen ("Szenarien")

7. Verteilungssicht

Deployment: Auf welcher Hardware werden die Bausteine betrieben?

8. Querschnittliche Konzepte und Muster

Wiederkehrende Muster und Strukturen.

Fachliche Strukturen.

Querschnittliche, übergreifende Konzepte, Nutzungs- oder Einsatzanleitungen für Technologien. Oftmals projekt-/systemübergreifend verwendbar!

9. Entwurfsentscheidungen

Zentrale, prägende und wichtige Entscheidungen.

Dokument



Microsoft
Word-Dokument

Legende:

anforderungsbezogene
Informationen

Strukturen der Lösung (Sichten)

übergreifende (technische)
Informationen

besonders wichtige Entscheidungen

Was für Aufgaben hat ein Software-Architekt?

*The architect allows things to happen. He shapes events as they come.
He steps out of the ways and let the design speak for itself.*
(nach P. Kruchten)

*garantieren klären beweisen entwerfen
entscheiden balanzieren kommunizieren
beraten dokumentieren implementieren
bewerten vereinfachen managen*

*Das Leben von Software-Architekten besteht aus einer langen und
schnellen Abfolge von Entwurfsentscheidungen, die meist im Dunkel
getroffen werden. (nach P. Kruchten)*

Architektur auf verschiedenen Ebenen

- **Architektur im kleinen:** Architektur individueller Programme – die Art, wie ein individuelles Programm sich in Komponenten aufteilen läßt.
- **Architektur im großen:** Architektur komplexer Unternehmenssysteme, die andere Systeme, Programme und Programm-Komponenten enthalten. Diese Unternehmenssysteme sind verteilt über verschiedene Computer, die unterschiedlichen Unternehmen gehören und von diesen gemanaged werden.

Wiederverwendung von Architektur

- **Systeme aus demselben Bereich** haben oft ähnliche Architekturen, die Eigenheiten dieses Bereichs widerspiegeln.
- **Software-Produktlinien** werden gebaut auf einer Kernarchitektur mit Varianten, die Anforderungen unterschiedlicher Kundengruppen erfüllen.
- Die Architektur eines Systems kann basierend auf einem **Architekturmuster** oder „**Stil**“ entworfen werden
 - Solche Muster oder Stile fangen das wesentliche einer Architektur oder eines Architekturaspekts auf eine wiederverwendbare Weise ein

Zusammenfassung

- Software-Architektur beschreibt, wie ein System organisiert ist
- Software-Architektur ist unterschiedlich darstellbar und aus unterschiedlichen Perspektiven betrachtbar
- Architektur hat eine wichtige Brückenfunktion zwischen Anforderungen und Implementierung
- Architektur unterstützt beim Umgang mit Komplexität



Literatur: Ergänzung

Was hat man als Entwickler von Architektur?

D. Spinellis: *Architecture from a Developer's Perspective*. IEEE Software vol.32 No. 05, 2015

<http://bit.ly/1F5T0hY>